

Instrucciones

- Lea con detenimiento y desarrolle **individualmente** cada una de las actividades a realizar durante la experiencia.
- Cree un archivo con extensión **.cpp** con lo desarrollado. El nombre del archivo debe tener el siguiente formato: **TEL102_C2_Nombre_Apellido.cpp** (Ej. TEL102_C2_Patricio_Olivares.cpp), sin incluir tildes.
- Las evaluaciones de la asignatura deben ser realizadas de manera **presencial** según lo estipulado en las reglas de clases y evaluaciones de la asignatura.
- Enviar el archivo a través de la página de aula del ramo, sección “Control 2” hasta las 10:50:00 del día Martes 01/10/2024 hora local continental de Chile.
- Utilice herramientas conocidas o aprendidas en clases. **No copie literalmente de recursos online.**
- La evaluación es de carácter individual. Está **estrictamente prohibido** tanto la comunicación entre estudiantes, así como el uso de herramientas generativas tales como chatGPT, Github copilot, y similares.
- Sea riguroso con las instrucciones de desarrollo.
- ¡Éxito!

1. IntergalacticTech - Gestión de Equipos de Comunicación en Misiones Espaciales

La empresa de tecnología espacial **IntergalacticTech**, encargada de disponer de los sistemas de comunicación para viajes interestelares, necesita un sistema confiable para la gestión de sus equipos. Para ello, han solicitado sus servicios como programador.

Al necesitar un sistema flexible, el cliente le exige que este sistema utilice **memoria dinámica** para registrar las misiones y sus equipos. También se exige la liberación correcta de cualquier elemento en memoria dinámica al finalizar el programa para evitar fugas de memoria.

El programa debe realizar las siguientes tareas (recuerde que puede utilizar las funciones `std::cin.getline()` y `std::cin.ignore()`, puede consultar un ejemplo de uso en el siguiente [enlace](#)):

- Registrar la información básica de una misión, incluyendo el nombre de la misión y los equipos de comunicación asociados.
- Asignar equipos de comunicación, almacenando su nombre y frecuencia operativa (en GHz).
- Mostrar la información completa de la misión y de los equipos asignados.

- Liberar toda la memoria asignada dinámicamente al finalizar el programa.

Debe utilizar como base para su código el presente en el archivo *c2par200.cpp*, disponible en la página de Aula.

c2par200.cpp

```
#include <iostream>

// Estructura para almacenar la información de un equipo de comunicación
struct equipo {
    char nombre[100]; // Nombre del equipo (ej. Antena Satelital)
    float frecuencia; // Frecuencia operativa (en GHz)
};

// Estructura para almacenar la información de las misiones espaciales
struct mision {
    char nombre[100]; // Nombre de la misión (ej. Misión Lambda)
    int n_equipos; // Número de equipos de comunicación asignados
    equipo* equipos; // Arreglo dinámico de equipos
};

// Prototipos de las funciones
mision* registrarMision();
void asignarEquipos(mision& m);
void mostrarMision(mision& m);

int main() {
    mision* misionActual = registrarMision();
    // Ojo, se derreferencia antes de pasar la misión!
    asignarEquipos(*misionActual);
    mostrarMision(*misionActual);

    // Complete aquí el código main necesario

    return 0;
}

// Escriba aquí el código de las funciones solicitadas
```

1. (30pts) Implemente la función `mision* registrarMision()`, que permita a un usuario registrar una nueva misión ingresando datos por consola. Esta función debe:

- Crear una nueva misión en memoria dinámica (heap).
- Solicitar y registrar los datos de la misión ingresados por terminal, incluyendo nombre de la misión y número de equipos de comunicación.
- Crear un arreglo de equipos en memoria dinámica (heap) y asociarlo a dicha misión.

Salida esperada:

Salida (consola)

```
Registrando nueva misión.
Ingrese nombre de la misión: Misión Lambda
Ingrese cantidad de equipos de comunicación: 2
```

2. (30pts) Implemente la función `void asignarEquipos(mision& m)`, que recibe una misión `m` a la cual se le pueden asignar equipos de comunicación por terminal. Esta función debe:

- Crear la cantidad de equipos de comunicación asociados a la misión.

- Por cada equipo, solicitar por consola al usuario tanto el nombre del equipo como su frecuencia operativa, y almacenarla en el equipo creado.
- Agregar dicho equipo en la misión m.

Salida esperada:

Salida (consola)

```
Asignando equipos para misión: Misión Lambda

Ingrese nombre del equipo: Antena Satelital
Ingrese la frecuencia operativa (GHz): 2.4
¡Equipo asignado!

Ingrese nombre del equipo: Transmisor Láser
Ingrese la frecuencia operativa (GHz): 5.8
¡Equipo asignado!
```

3. (25pts) Cree la función `void mostrarMision(mision& m)`, que permita mostrar toda la información de una misión y sus equipos de comunicación por consola. Esta función debe:

- Mostrar el nombre de la misión y el número de equipos de comunicación asignados.
- Mostrar toda la información de cada uno de los equipos asignados a la misión.

Salida esperada:

Salida (consola)

```
Mostrando información para misión: Misión Lambda

Número de equipos: 2
Equipo Antena Satelital, frecuencia de operación (GHz) 2.4
Equipo Transmisor Láser, frecuencia de operación (GHz) 5.8
```

4. (15pts) Complete el código de la función `main()` para liberar todos aquellos elementos memoria dinámica (heap). Recuerde que puede utilizar `valgrind` para verificar que no hay problemas de **memory leak** o de **dangling**.

Ej. de salida del programa completo:

Salida (consola)

```
Registrando nueva misión.
Ingrese nombre de la misión: Misión Lambda
Ingrese cantidad de equipos de comunicación: 2

Asignando equipos para misión: Misión Lambda

Ingrese nombre del equipo: Antena Satelital
Ingrese la frecuencia operativa (GHz): 2.4
¡Equipo asignado!

Ingrese nombre del equipo: Transmisor Láser
Ingrese la frecuencia operativa (GHz): 5.8
¡Equipo asignado!

Mostrando información para misión: Misión Lambda

Número de equipos: 2
Equipo Antena Satelital, frecuencia de operación (GHz) 2.4
Equipo Transmisor Láser, frecuencia de operación (GHz) 5.8
```